

WAYSCONF 2026 · MASTERCLASS · 90 MINUTES

Build an AI that checks and fixes its own work

Agentic loops in practice.

Szymon Paluch · CTO, Susteen

ROOM-PM · 16 September 2026

0:00 – 0:08

The case

Most AI demos stop here

A prompt goes in. Something impressive comes out. Everyone claps.

Then someone asks a question the demo was not built to survive:

Does it match the design?

Is the contrast legal?

Does it work on a phone?

Is that the actual copy, or copy it invented?

no answer

This is a talk about the word *no*

A loop is only worth building if something in it can refuse the work without asking a language model for an opinion.

What you will have at 16:25

A landing page for a music festival that does not exist, built from a Figma design by an agent you instructed, checked by a verifier that does not care what the agent thinks, fixed by the agent when it failed, and deployed to a URL you can send to someone.

all gates green

```
npm run check
```



✓ 9/9 gates passed.

Build	PASS
Runtime	PASS
Structure	PASS
Accessibility	PASS
Design tokens	PASS
Visual fidelity	PASS
Content	PASS
Links	PASS
Lighthouse	PASS



The loop

0:08 – 0:15

Checkpoint 0

Everyone gets a working environment

Two doors, same room

On your machine

```
gh repo create my-turbine \
  --template hculap/waysconf-2026-agentic-loops \
  --public --clone
cd my-turbine
npm install
npm run dev
```



Node 20.11 or newer. Nothing else.

In your browser

Use this template → Create a new repository

Code → Codespaces → Create codespace

Three minutes. Node, both agents and the browser the gates need are already installed.

No install. No admin rights. Works on a locked-down work laptop.

GitHub, for people who have never used it

YOU ALREADY KNOW THIS AS

GITHUB CALLS IT

A shared folder that remembers every version

a **repository**

Save, with a note about what changed

a **commit**

Duplicating a file so you can experiment safely

a **branch**

Sending your version back for review

a **pull request**

The folder your website is published from

the thing Netlify reads

For the next ninety minutes you need the first two. That is genuinely all.

The terminal, in one slide

```
pwd          # where am I
ls           # what is here
cd my-turbine # go in there
npm run dev  # start the site
```



Ctrl+C stops whatever is running.

It is a text field that runs programs.

It gives you no feedback while it works, which reads as "broken" and is almost always "busy".

When a command finishes you get the prompt back. When you do not get the prompt back, it is still going.

Full version: [docs/TERMINAL-IN-TEN-MINUTES.md](#)

0:15 – 0:25

Design in, not screenshots in

MCP, in one sentence

A standard way for a coding agent to call tools that are not inside it.

Two ways to hand over a design

A screenshot

The agent looks at pixels and infers.

The orange becomes *an* orange.

The spacing becomes *some* spacing.

It looks roughly right.

token gate fails

The design itself

Colours arrive as values. Spacing arrives as numbers.

```
"color": {  
  "accent": {  
    "sodium": { "$value": "#FF6A1A" }  
  }  
}
```



Nothing is guessed, so nothing is guessed wrong.

token gate passes

The honest bit about Figma

The official Figma MCP server needs a paid seat. On a free account it connects and then declines to hand anything over.

So the repository ships the design as data

```
design/export/  PNGs at 390, 768, 1440
design/tokens/  every colour, size, space
design/FIGMA-SPEC.md  the layout, in numbers
```



Nothing in this workshop is blocked by a free Figma account.

And as a plugin that rebuilds the file

```
figma-plugin/
```



Run it inside Figma and it constructs the whole TURBINE file — variables, text styles, components, every frame — from the same tokens the code uses.

0:25 – 0:40 • SPRINT 1

Plan first, then build

Read the plan before it writes the code

Plan mode is the cheapest correction in the entire loop. A wrong assumption costs one sentence to fix here, and twenty minutes of generated code to fix later.

```
# Claude Code: Shift+Tab until the footer says "plan mode on"  
# Codex CLI: ask for a plan and refuse to accept code until you have read it
```



Then read it. Actually read it. The plan is where you find out it misunderstood the brief.

Your turn — 15 minutes

```
# 1. point your agent at the repo
claude          # or: codex

# 2. give it the first prompt
loop/prompts/01-plan.md

# 3. read the plan, then let it build
#   the nav and the hero
```



Falling behind is normal and planned for.

```
git checkout step-1
```



Every stage of this session is a branch.
CHECKPOINTS.md lists all six.

0:40 – 0:55

The verifier

The only part that matters

A real loop and a fake one

Where self-assessment is fine, and where it is not

Fine

Is this sentence any good.

Which of these five findings matters most.

Does this read as the same brand.

Is the hierarchy clear.

Taste, prioritisation, judgement.

Not fine

Is this correct.

Is it accessible.

Does it match the design.

Is it finished.

Anything with an external truth.

Use the external truth.

Three tiers of checking

Nine gates, 55 criteria, no opinions

GATE	WHAT IT DECIDES	CRITERIA
Build	It compiles, no type errors, one page out	AC-01...05
Runtime	No console errors, no failed requests	AC-03...04
Structure	Sections, headings, landmarks, skip link, ids	AC-06...14
Accessibility	axe clean at three widths, focus, keyboard, motion	AC-15...25
Design tokens	Every colour and size came from the design	AC-26...33
Visual fidelity	Pixels match the design, within a stated budget	AC-34...37
Content	The real copy, all twelve artists, no placeholders	AC-38...47
Links	Nothing points at nothing	AC-48...51

Watch it catch something real

The palette has a trap in it, on purpose.

```
color.text.muted #6B7280  
color.bg.base #0A0B0D
```



4.07 : 1

Under the 4.5:1 that body text requires.

```
color.text.secondary #A7AEBB  
color.bg.base #0A0B0D
```



8.9 : 1

Ask any model for "a muted grey for supporting copy" and watch which one it reaches for.

The failure report is the interface

```
**3. [AC-28] color.text.muted (#6B7280) used outside the footer legal block**
```

- Where: `section[data-section="lineup"] .artist-card p`
- Expected: `color.text.secondary (#A7AEBB)`
- Actual: `#6B7280` – 4.07:1 against #0A0B0D, minimum 4.5:1
- Hint: supporting copy uses text.secondary; muted is legal text only



No model wrote that. A program measured it and printed it.

The agent's next iteration starts by reading this file.

The rule that holds it all up

The agent may never edit the verifier.

It is written into `AGENTS.md`. It is checked in the dry-run harness. If an agent modifies anything under `checks/`, the run is recorded as compromised regardless of what the gates then say.

```
| Verifier modified by the agent | no |
```



0:55 – 1:10 · SPRINT 2 RUNS IN THE BACKGROUND

The loop, and workflows

Start your loop now, then listen

```
bash loop/ralph.sh
```



It will run for several minutes. Leave it. I will talk over it, and when it stops we will look at what it did.

```
AGENT=codex bash loop/ralph.sh # if you are on ChatGPT Plus  
MAX=3 bash loop/ralph.sh      # if you would rather it stopped sooner
```



The whole loop

```
for i in $(seq 1 "$MAX"); do

  # the verifier runs FIRST: if the gates are green there is nothing to do,
  # and an agent asked to improve passing work will invent a reason
  if npm run --silent check; then
    echo "All gates passed on iteration $i."; exit 0
  fi

  # hand the report back, unchanged
  claude -p "$(cat loop/PROMPT.md)" --permission-mode acceptEdits

  # a commit per pass, so you can see exactly what each one did
  git add -A && git commit -q -m "loop: iteration $i"
done
```



That is it. The rest of `ralph.sh` is guard rails.

Why a fresh context beats a long one

One long conversation

Forty minutes of history.

Three abandoned approaches still in context.

The original brief, sixty thousand tokens ago.

Reasoning gets worse as it fills.

A fresh context each pass

Reads `checks/report.md` — the current failures.

Reads `loop/PROGRESS.md` — what was already tried, and what it cost.

Reads the brief.

Notes on disk beat memory in context.

When one agent is not enough

Adversarial review

The instruction is not "review this".

The instruction is **"refute this, and default to refuted if you are unsure"**.

```
const votes = await parallel(LENSES.map(lens => () =>
  agent(`Try to refute: ${finding}. Default to refuted if uncertain.` ,
    { schema: VERDICT })))

const survives = votes.filter(v => !v.refuted).length >= 2
```



A reviewer asked to find problems finds problems. A reviewer asked to destroy a specific claim either destroys it or fails to.

1:10 – 1:22 • SPRINT 3

Ship it

Deploy

```
npm run build  
npx netlify deploy --prod --dir=dist
```



It will ask you to sign in, then to pick or create a site. Take the URL it prints.

Put your URL on the board. We are going to look at all of them.

1:22 – 1:30

What this does not do

Honesty

THE CLAIM

The a11y gate is green

The pixel diff passes

Adversarial review found nothing

All 55 criteria pass

THE LIMIT

Automated rules reach maybe 30–40% of real WCAG failures

It saw no change. It has no idea whether a change would be better

Models checking models. Least reliable tier, never the only one

Nothing here has an opinion about whether the design is good

Green means **no known defect**. It has never meant finished.

Where loops actually fail

It edits the test. The gate goes green and nothing was fixed. Forbid it in writing, and check that the file did not change.

It oscillates. Two wrong states, alternating. Usually two criteria that contradict each other. Cap the iterations and read the log.

It keeps going after it is done. Run the verifier *first*, every pass.

It runs out of context mid-task. Put the state in a file.

It reports success without running anything. Only believe the exit code.

It hangs. Not failing, just waiting. Every step needs a deadline — this repository lost six minutes to exactly that, and it is written down.

Take this to work on Monday

Do not start with a whole landing page. Start with one gate.

1. Pick the check your team already does by hand and resents.
2. Make it a program that exits 0 or 1.
3. Put the failure output somewhere an agent can read.
4. Only then put an agent in front of it.

The loop is the easy part. The oracle is the work.

EVERYTHING FROM TODAY

TURBINE

`github.com/hcūlap/waysconf-2026-agentic-loops`

The brief · every prompt I used · the verifier · the Figma plugin ·
the evidence from both clean-room runs · these slides

Questions. And then go and put your URL on the board.